

Full Audit — Things To Check

A fresh read of the whole mobile app: Flutter session flow, BLE, the native Kotlin engine, CSV recording, report maths, database and Android configuration. Every item below is something found in the code that looks wrong, risky, or needs a decision. **Nothing here has been reproduced on a device.** Treat each one as a claim to verify, not a fact. Some will turn out to be intentional — say so and we close them.

4 September 2026 · audited `wergonic-flutter` · `dev` · `c9ec2c4` (1 Sep 2026)

Scope: `lib/` (1,117 Dart files), `android/app/src/main/kotlin/` (40 Kotlin files), manifest and Gradle config.

P0 = can lose or corrupt session data · P1 = wrong behaviour a user would notice · P2 = smaller / cleanup / decision needed

14

P0 — DATA RISK

21

P1 — WRONG BEHAVIOUR

22

P2 — SMALLER

6

AREAS

READ THIS FIRST

Three things stand out above everything else.

- 1. The app has no Android foreground service.** It records for hours over BLE with nothing telling Android to keep the process alive. The wakelock keeps the screen on while the app is in front, but the moment it is backgrounded there is nothing protecting it. This is the classic cause of "the session just stopped" on Samsung and Xiaomi devices.
- 2. Files are routed by searching the file path for words like "Arm", "hand", "trunk", "head" and "hr".** The path contains the workplace name and the worker's name. A client called *Handelsbanken*, *Armatec* or *Head Office*, or a worker called *Christoffer*, gets their data parsed as the wrong body part or written with the wrong headers. There is already a comment in the code saying this exact pattern was fixed in one place — it was not fixed everywhere.
- 3. One transient BLE error can silently disable a sensor for the rest of the app run.** If a subscription gets three 503s in a row (which the code's own comment says happens under radio contention), that sensor is marked "unsupported" for the whole process and records nothing — for that session and every session after it, until the app is restarted. Only a log line says so.

A · Recording & data loss 14 items

Anything here can mean a session comes back with missing or short data.

A1 No foreground service — Android can kill the app mid-session P0

The manifest declares no service at all, and there is no `FOREGROUND_SERVICE`, `FOREGROUND_SERVICE_CONNECTED_DEVICE`, `WAKE_LOCK` or `POST_NOTIFICATIONS` permission. A multi-hour BLE recording with the app in the background is subject to Doze, App Standby and OEM battery killers. Check what actually happens when the user presses Home or turns the screen off mid-session on a Samsung device.

`android/app/src/main/AndroidManifest.xml`

A2 Three 503s permanently disable a sensor stream for the whole app run P0

`DataSubscriber` retries a 503 three times, then adds the sensor+stream to an `unsupported` set that is never cleared — deliberately, per the comment, and it survives session restarts. But 503 also happens transiently under radio contention. Once it triggers, that limb records nothing for every session until the app is restarted, with no UI signal.

`android/.../ble/data/source/transport/DataSubscriber.kt`

A3 `waitForQueueEmpty` returns instantly after the first time it is used P0

`_writeQueueEmpty` is a single `Completer` created once per writer and completed the first time the queue drains. Every later call — including the one in `closeAll()` at session end — returns immediately even with batches still queued, and then the sinks are closed. Verify what happens when the queue is non-empty at finish.

`core/services/csv_recording/writing/limb_csv_writer/wait_for_queue_empty.dart`

A4 CSV error reporting is dead from the second session onward P0

`CsvWriterManager` is a lazy singleton. `closeAll()` closes `_globalErrorController`, which is a `final` field and never recreated. The next session subscribes writers to a closed controller, so write errors either throw or vanish — and the "Data save error" snackbar never fires again until the app restarts.

`core/services/csv_recording/writing/csv_writer/csv_writer_manager.dart + close_all.dart`

A5 Legacy CSV fallback loses the last buffer P0

If `initializeCsvWriterSystem()` throws, writers fall back to the legacy buffer, which only writes when it overflows. At finish, `stopCsvWriting()` runs before `stopReadings()` and `stopWriting()` clears `writeBuffer` — so the flush that follows has nothing left. Up to a minute of data per limb, silently. `flushLegacyBuffer()` exists but nothing calls it.

`sessions/session/domain/usecases/data_processing/write_data_to_csv.usecase.dart`

A6 Native engine events are dropped when the buffer fills P0

The Kotlin engine emits with `tryEmit()` on a `SharedFlow` with 64 slots and no overflow strategy. `tryEmit` returns false and drops the event silently when there is no room, and the return value is ignored. The Dart side already has a workaround for this (`_reconcileConnectedFromRssi` repairs the map when a `CONNECTED` event goes missing) — the root cause is here.

`android/.../ble/data/session/SensorStateMachine.kt`

A7 One throwing RSSI read kills RSSI sampling for the whole process P0

`batteryLoop()` wraps its work in `try/catch`. `rssiLoop()` does not — the `try/finally` in it is only for diagnostics. If `rssiReader.read()` or `scheduler.withRadio` throws, the coroutine ends and never restarts. RSSI stops, and so does the reconnect self-heal that depends on it.

`android/.../ble/data/session/SensorStateMachine.kt - rssiLoop()`

A8 CSV write failures are effectively invisible P1

`sink.write()` is buffered and does not throw on failure — errors surface later on flush or close. But `_totalSamplesWritten` is incremented right after the `write()` call, inside a `try/catch` that will rarely fire. So `rows_written` in the integrity manifest can report rows that never reached disk.

`core/services/csv_recording/writing/limb_csv_writer/process_write_queue.dart`

A9 A session killed while recording is never recovered P1

The splash sweep only picks up sessions marked `finishing` — i.e. the app died during close-out. A session killed while still recording has no finish state at all, so it is never swept, never marked, and the user is never told. Its `endedAt` stays null, so it shows a duration of 00:00:00.

`sessions/session/domain/usecases/flow/sweep_interrupted_sessions.usecase.dart`

A10 Zip files from the auto-upload path are never deleted P1

`syncWorkSession` deletes the zip on both success and failure. `uploadSessionFiles` (the automatic path) never does. They accumulate in `/zips`. Worse: `createZipForSessionData` reuses any existing non-empty zip at the same path, so a stale zip from an earlier failed attempt gets uploaded again instead of a fresh one.

`sessions/session_sync/data/repositories/work_session_sync.data.repository.dart`

A11 Ceiling and completion can both settle the same session, out of order P1

In `SessionFinalizerCubit._run`, when the 120 s ceiling fires it calls `_settle(failed)` while the real sequence is still running. When that sequence lands it calls `_settle()` again. Both are async DB writes with no ordering guarantee, so the final stored state is a race. The intent (self-heal back to closed) is right; the sequencing is not guaranteed.

`sessions/session/presentation/logic/cubits/finalizer/session_finalizer_cubit.dart`

A12 Recording page teardown runs while the close-out is still draining P1

`_finishSession` starts the close-out unawaited, then navigates. The page's `dispose()` then calls `stopAllStreams()` — concurrently with the finalizer's `closeFiles` step. The comment says this is safe; worth confirming no sample in flight is lost when the two overlap.

`sessions/session/presentation/pages/recording/work_session_recording.page.dart`

A13 Sinks are reused for existing files without checking the header shape P1

`getOrCreateSink` opens an existing file in append mode and skips header writing entirely. If the row shape changed (for example vibration detection toggled between the file being created and reopened), rows with a different column count get appended under the old header, and the whole file then fails the report's exact column-count check.

`core/services/csv_recording/writing/file_handle_manager/get_or_create_sink.dart`

A14 Queue overflow drops the oldest batch with only a warning P2

When the write queue hits `maxQueueDepth` (10 buffers) the oldest is discarded. The comment says "Maximum 10 minutes of buffered data" but it is 10 *buffers*, each `bleDataBufferSize` samples — not minutes. Worth confirming the real capacity, and whether dropping the oldest rather than the newest is the right choice.

`core/services/csv_recording/writing/limb_csv_writer/limb_csv_writer.dart`

B · Report numbers 14 items

These change the values that end up in front of the client.

B1 CSV files are routed to the wrong parser by name matching P0

The report picks the processor by searching the *full file path* for `Arm` (case-sensitive), then `hand`, `trunk`, `head`, `hr` — in that order. The path contains the workplace folder and the worker's name twice. A workplace called *Armatec* or *Handelsbanken*, or one called *Head Office*, sends every limb file to the wrong processor and produces a report built from the wrong columns. Because `Arm` is checked first, an "Arm" hit wins over everything else.

`core/services/report/process_file_in_isolate.dart`

B2 Header type is picked by `path.contains('hr')` with no limb guard P0

`FileHandleManager._writeHeaders` writes HR headers to any file whose lowercased path contains "hr" — which includes *Christoffer*, *Ahrén*, *Ehrling*. The same decision in `createCSVForSession` is written correctly (it checks `limb == null` first), and `CsvRowShape` carries a comment explaining exactly why the filename must not be trusted. This path was missed. It fires whenever a limb file did not already exist — e.g. a sensor connected after the session was created.

`core/services/csv_recording/writing/file_handle_manager/file_handle_manager.dart`

B3 Break time is recorded into the CSV and counted as posture P0

Pausing a session does not stop samples reaching the writers: `mode` stays `recording`, and `CsvWriterManager.pauseSession()` only cancels the flush timer — `addData` still queues rows. On top of that, `pauseCsvWriting()` is gated behind the `useNewCsvSystem` setting, which defaults to `false`, so it is usually a no-op anyway. The report's angle calculations read every row in the file with no break filter.

`measurements/.../measurements_manager_cubit.dart` + `csv_writer/pause_session.dart`

B4 Head and side-bending do not filter sensor dropouts — trunk and arm do P0

`calculateTrunkBendingRisk` and `calculateArmElevationAngleTimePercentage` both drop readings whose duration exceeds `dropoutGapMs`, with a comment saying the report must count the same time as the live view. `calculateHeadBendingRisk` and `calculateTrunkSideBendingRisk` have no such filter. So in the same report, head and side-bending count sensor-absence gaps as posture and trunk/arm do not.

`core/utils/calculations_utils.dart`

B5 Trunk side-bending times are doubled on purpose P1

`processTrunkCsv` adds every side-bending value to the list twice, with a comment saying not to deduplicate it because it matches the backend. Percentages are unaffected, but the reported *durations* are exactly 2× reality. Confirm whether the displayed side-bending times are meant to be doubled, or whether the backend has the same bug and both need fixing.

`core/services/report/process_trunk_csv.dart`

B6 Angular velocity uses a 30 threshold but the fields are called under60/over60 P1

`calculateAverageGeneralizedAngularVelocity` splits at `velocity < 30` and writes the result into fields named `under60` and `over60`. Either the threshold or the field names is wrong. Whichever it is, the report label and the number probably disagree.

`core/utils/calculations_utils.dart`

B7 Angular velocity has no dropout filter either P1

Same function as B6. Unlike arm elevation, it does not exclude readings longer than the dropout gap, so a sensor drop is counted as low-velocity time and pulls the average down.

`core/utils/calculations_utils.dart`

B8 Total session duration is computed and then thrown away P1

`terminateSession()` carefully calculates `totalDuration` and passes it to `updateWorkSessionStatus`, which declares the parameter and never uses it. Nothing reads it anywhere. Either wire it up or delete it — but first check whether the server is supposed to be receiving it.

`sessions/session/presentation/logic/cubits/ongoing_work_session/work_session_cubit.dart`

B9 Local `startedAt` is set after the API call, so it differs from the server's P1

`startWorkSession` sends `started_at` to the API, then afterwards overwrites the local session with `DateTime.now()` — which is now later by however long the request took. The on-device duration is short by that amount, and the two records disagree. On a slow connection this is seconds.

`sessions/session/presentation/logic/cubits/ongoing_work_session/work_session_cubit.dart`

B10 Report parsing keys off a hardcoded column count that differs by build flavour P1

Rows are only counted when `row.length` exactly equals a constant — and that constant is 17 in dev and 15 in prod, because dev adds two extra columns. Any mismatch silently skips every row and produces an empty report with no error. The header row is right there; the indices should come from it, not from a constant. Also: `handCsvRowLength` is defined but never used.

`core/config.dart` + `core/services/report/process_*_csv.dart`

B11 No guard for a missing column — `row[-1]` crashes the report P1

`headers.indexOf(...)` returns -1 when a column is absent. Only `processHeadCsv` checks one of them (`rollColumnSideBendingIndex`). Trunk, arm and hand do not check any, and hand's `row.length > index` guard passes trivially for -1. Any header drift throws a `RangeError` inside the isolate.

`core/services/report/process_arm_csv.dart`, `process_trunk_csv.dart`, `process_hand_csv.dart`

B12 Report rows are read as `dynamic` with no type check P1

The CSV parser returns `int` for a whole number and `double` otherwise. The values go straight into `(double, int)` records. If any angle is ever serialised without a decimal point, the record construction throws a `TypeError` mid-report. Worth a defensive `num.toDouble()`.

`core/services/report/process_*_csv.dart`

B13 `calculatePercentiles` throws on an empty list P2

With an empty input the index goes negative and indexing throws. Every current caller guards it, but the function itself does not — and it is a shared utility. One line to make it safe.

`core/utils/calculations_utils.dart`

B14 HR risk has no guard on resting heart rate P2

`hrCapacity = (220 - age) - restingHr`. Age is clamped to 1–119 but resting HR is not validated at all. A resting HR of 0 (not entered) or one above max produces a nonsensical or negative capacity, and every threshold derived from it is wrong. Check what the app does when the field is left blank.

`core/utils/calculations_utils.dart` — `calculateHrRisk`

C · Session & work cycle flow 12 items

C1 Work Cycle sessions never auto-finish at the time limit P1

The timer listener does `isWorkCycle ? switchTask(...) : checkMaxSessionTime(...)`. In Work Cycle mode the max-session check is never called, so `sessionDurationLimit` (default 8 h) is not enforced and the session runs until someone stops it.

`sessions/session/presentation/widgets/recording/session_auto_control.widget.dart`

C2 A failed work cycle event stops auto-advance for the rest of the session P1

`_isCreatingEvent` is set to `true` before creating the event and reset only by the `onComplete` callback, which fires inside the happy path. If `createWorkCycleEvent` throws, the catch block does not reset the flag — so `switchTask` is blocked forever and tasks stop advancing automatically.

`sessions/session/presentation/widgets/recording/session_auto_control.widget.dart`

C3 Duplicate-task race: `onComplete` fires before the index advances P1

In `createWorkCycleEvent`, `onComplete?.call()` (which clears `_isCreatingEvent`) runs *before* `await moveToNextTask()`. The 1-second timer tick can fire in that window and create a second event for the same task index.

`sessions/session/presentation/logic/utils/work_cycle_event_helper.dart`

C4 Work cycle task switching loses the network round-trip as unaccounted time P2

The event's `endTime` is stamped before `addSessionEvent`, and the next stint's start is stamped after it returns. The gap between them is the request latency, and it is attributed to nothing. Over a long cycle with many switches this accumulates.

`sessions/session/presentation/logic/utils/work_cycle_event_helper.dart`

C5 The session timer resets on every work cycle task P1

`resetTimer()` is called on each task change and again when tasks are re-fetched. So in Work Cycle mode what is displayed and used as "session duration" is really "time since the last task change". It is also what the feedback warm-up countdown reads. Confirm which behaviour is intended and label it accordingly.

`work_cycle_event_helper.dart + session_auto_control.widget.dart`

C6 The session timer keeps running through breaks P1

`SessionTimerCubit.pauseTimer()` and `resumeTimer()` exist but **nothing calls them**. The pause flow stops CSV writing and the work task stopwatch, but not the session timer. If break time is meant to sit inside the total that is fine — but the feedback warm-up countdown also runs off this timer, so a break counts toward "feedback in M:SS".

`sessions/session/presentation/logic/cubits/session_timer/session_timer_cubit.dart`

C7 "Calibration is stale because a sensor dropped" can never trigger P1

`LastUnintentionalDisconnectAt` is only written in `listenToDevicesStream`, the old MDS path. With `useNativeEngine = true` that stream never emits a disconnect — it only fires on connect, reconfigure and unpair. So the field stays null and `CalibrationReuseVerdict.disconnectedSinceCalibration` is unreachable: Quick Start will happily reuse a calibration taken before a sensor fell off and was put back on.

`bluetooth/.../sensor_connection_cubit.dart + measurements/domain/entities/calibration_reuse.dart`

C8 A devices push in `dispose()` can race the completed-status write P2

`SessionBlocListeners.dispose()` flushes a pending device-list update if the session has not been marked finished. Because the close-out runs in the background, `sessionTerminated` may not have arrived yet — so a write can land against a session that is being completed at the same moment.

`sessions/session/presentation/widgets/recording/session_bloc_listeners.widget.dart`

C9 Lite mode can record a session with zero limbs P1

In `updateWorkSessionStatus`, if lite mode is on and either `deviceInfo` or `selectedLimb` is null, the connected devices list is set to empty. Since there are no paired BLE devices in lite mode either, `missingLimbs` comes out empty too — so the session reports neither recorded nor missing limbs, silently.

`sessions/session/presentation/logic/cubits/ongoing_work_session/work_session_cubit.dart`

C10 The Firestore live-session document is never marked completed P1

`updateWorkSessionStatus` writes to Firestore only when the status is *not* completed; completion goes to the REST API instead. So the live document keeps whatever state it last had. Check whether the web panel's live view leaves finished sessions showing as still recording.

`sessions/session/data/sources/remote/session.remote.datasource.dart`

C11 ended_at falls back to "now" on a late retry P2

When the completed-status write runs and `session.endedAt` is null, the code sends `DateTime.now()`. If that write is retried later — after backoff, or via the retry-finish path on the summary — the server records the retry time as the session end, not the real one.

`sessions/session/data/sources/remote/session.remote.datasource.dart`

C12 Pause and resume call `setState` after awaits with no `mounted` check P2

Both `onPauseWorkSession` and `onResumeWorkSession` await several async calls and then call `setState` unguarded. If the page is disposed in between (auto-finish, navigation) that throws. Also, `pauseCsvWriting()` is awaited but `resumeCsvWriting()` is not — worth making symmetric.

`sessions/session/presentation/pages/recording/work_session_recording.page.dart`

D · What the user sees 7 items

D1 A limb that stops recording is announced by a 2-second snackbar, in English P1

Sensor data timeout, stream error, CSV write error and retry attempts are all surfaced as a 2-second snackbar with a hardcoded English string — in an app localised to five languages. After three failed retries the app gives up silently and keeps recording with nothing from that limb. There is no persistent indicator anywhere that a limb has stopped.

```
sessions/session/presentation/widgets/recording/session_bloc_listeners.widget.dart
```

D2 Shaking the phone opens the "report a problem" sheet — during recording P1

`ShakeDetector.autoStart` is registered globally in `_AppState` with a 5-shake threshold, so it is live on every screen including the recording page. The whole point of the app is that the worker is moving. Check whether this can fire mid-session and cover the screen — and consider disabling it while recording.

```
app.dart
```

D3 Exposure colours can go red in the first seconds of a recording P1

`PostureExposureTracker._project` extrapolates whatever has been measured so far to a full 480-minute working day, with no minimum sample requirement. Ten seconds spent above 60° projects to the entire day above 60°. There is a warm-up countdown widget, but it only hides the panels — it does not stop the tracker counting or the projection being unstable.

```
sessions/session/presentation/logic/utils/exposure/posture_exposure_tracker.dart
```

D4 The wakelock is never released P2

`WakeLockPlus.enable()` is called on entering the kiosk wrapper and there is no `disable()` anywhere in the app. Kiosk mode has a proper release guard; the wakelock does not. The screen will not sleep for as long as the app is in the foreground, including on Home and Settings after a session.

```
core/gesture_lock_wrapper.dart
```

D5 Session folders are named "Firstname-null" when there is no last name P2

`createPath` interpolates `"$firstName $lastName"` directly, so a null last name becomes the literal string `null` in the directory name. The *file* names use `fullName`, which handles it correctly — so the folder and the files inside it disagree.

```
core/functions/generate_path.dart
```

D6 Workplace name goes into the file path unsanitised P2

Same function. Only spaces are stripped. A workplace containing a slash creates nested directories; other characters can produce paths the zip or the report cannot find again.

```
core/functions/generate_path.dart
```

D7 The summary screen decides online/offline once and never updates P2

`_isOnline` is read in `initState` and stored in a `late final`. If the connection comes back while the user is on the summary, the screen still behaves as if offline.

```
sessions/session/presentation/pages/session_summary/session_summary.page.dart
```

E · Edge cases worth reproducing 10 items

E1 A break shorter than 2 seconds is counted as exposure P2

The exposure tracker discards a sample gap only if it exceeds `dropoutGapMs` (2 s). Pausing and immediately resuming leaves a sub-2-second gap that is counted as continuous posture at the pre-break angle.

`exposure/posture_exposure_tracker.dart`

E2 The exposure tracker records a frozen angle on non-reading states P1

`_listenForExposure` fires on every state a measurement cubit emits — including `error` and `dataTimeout` — and reads the last known angle each time with a fresh `DateTime.now()`. A sensor that has stopped sending but keeps emitting state changes can keep accumulating exposure at its last posture.

`measurements/.../measurements_manager_cubit.dart`

E3 Filters reset to zero on every stream restart P2

The Butterworth filters start from a zero state, so after every reconnect or stream restart the first handful of samples ramp up from zero instead of the true gravity vector — producing wrong angles for a fraction of a second. Check whether that shows as a spike in the live feedback or in the CSV.

`core/utils/butterworth_filter.dart`

E4 Sensor serial is derived by stripping non-digits from the device name P1

`BleDeviceEntity.serial` is `name.replaceAll(RegExp('[^0-9]'), '')`. Two sensors from different manufacturers with the same digits collide, and a device whose name has no digits gets an empty serial. Battery tracking, battery alerts and the Kotlin `isSerialConnected` check all key off this.

`bluetooth/domain/entities/ble_device.entity.dart`

E5 Breaks are detected by the literal English string "Work Break" P1

`breakDuration` matches `event.name == "Work Break"`. A custom activity named exactly that would be counted as a break, and if the label is ever localised or renamed on the backend all break time silently becomes work time. Give it an id or a flag instead of a name comparison.

`work_session.entity.dart`, `work_session_model.dart`, `session_timeline_builder.dart`

E6 Enums are persisted by `.name`, against the repo's own rule P1

`AGENTS.md` says never to persist an enum by `.name` or `.index` and to use an explicit `code` field. But `syncStatus`, `finishState`, `finishFailure`, `role`, `limb` and `battery` are all written to Drift by `.name`. Renaming any of those values silently orphans every stored row.

`core/database/daos/`

E7 No downgrade or corruption path for the local database P1

Schema is at 22 with no `onDowngrade` and no recovery if the file is corrupt. Since the app is offline-first and holds unsynced sessions, an unopenable database means unrecoverable data. Also worth checking: the upgrade chain has no branches for versions 4, 5, 6 and 17 — confirm those were genuinely covered by the surrounding steps.

`core/database/database.dart`

E8 The native engine never forgets a limb P2

`connections` and `macToLimb` in `SensorStateMachine` are only ever written, never removed — `disconnect()` just flips the status. So `ownsConnections()` stays true forever after the first pairing, and `resync()` replays entries for limbs that were unpaired. `setPhase` on an unknown limb also creates a placeholder entry with an empty MAC.

`android/.../ble/data/session/SensorStateMachine.kt`

E9 Dead legacy BLE path that conflicts with the live one P2

With the native engine on, `listenToDevicesStream`'s `disconnect` branch is unreachable — including its 30-second reconnect watchdog. If that path is ever re-enabled it will fight the 2-minute "offer a Stop" timer: the watchdog marks the limb disconnected at 30 s, and the stop offer only appears if the limb is still marked reconnecting. Either delete the dead branch or reconcile the two timeouts.

`bluetooth/.../sensor_connection_cubit.dart`

E10 The 208 Hz filter has a DC gain of 0.86 instead of 1.0 **P2**

All five sampling rates were checked numerically. 13, 26, 52 and 104 Hz all give a clean 5 Hz low-pass with unity DC gain. 208 Hz comes out at 0.861 — the coefficients look truncated. Angles are computed from normalised vectors so it should not affect them, but anything using absolute magnitude would be off by 14%. Confirm whether 208 Hz is used at all.

```
core/Utils/calculations_utils.dart - _blpCoefficients / _alpCoefficients
```

F · Cleanup & small stuff 10 items

F1 Auth interceptor has no error handling and no 401 retry P1

`AuthInterceptor.onRequest` calls `await user.getIdToken()` with no try/catch. If the token refresh fails (no network mid-refresh), the exception escapes `onRequest` instead of being turned into a rejection. There is also no 401 handling anywhere — no forced refresh, no retry, no sign-out.

`core/network/auth_interceptor.dart`

F2 All Dio errors are dropped before reaching Sentry P2

`beforeSend` returns null for every `DioException`, on the grounds that the backend reports them. That also hides client-side failures the backend never sees — timeouts, DNS failures, cancelled requests. Worth confirming this is still the right trade.

`main.dart`

F3 Debug noise left in a production code path P2

`initialize_session.dart` opens with sixteen lines of leftover `debugPrint` calls, several of them printing "CSV writing session closed" at the point where a session is being *opened*. Confusing in any log someone reads later.

`core/services/csv_recording/writing/csv_writer/initialize_session.dart`

F4 Dead code to remove P2

`handCsvRowLength` (never read), `_workTasksRemoteDataSource` in the sync repository (marked `ignore: unused_field`), `safeIndex` in `switchTask` (computed then ignored), `pauseTimer` / `resumeTimer` (never called), and the `totalDuration` parameter from B8.

F5 TODOs sitting in live paths P2

Eleven of them, including `//TODO : take a look at this` directly above `updateWorkSessionStatus` in the remote data source — the method that decides whether a session is marked completed. Worth triaging: each is either a real task or noise to delete.

F6 Typos in public names P2

`calculateAngularVilocity`, `nintiethPercentile`, `isFristStart`, `onDiconnect`, `chronologicallyOrdredEvents`, `rrCollumnIndex`. Cosmetic, but `nintiethPercentile` is a model field name that ends up in stored data and possibly in exports — check before renaming it.

F7 WorkTaskEntity.copyWith silently resets the stopwatch P2

`copyWith` builds a brand-new entity, and the constructor creates a fresh paused `Stopwatch`. Any `copyWith` on a running task loses its elapsed time. The class is also `Equatable` on `[id, name]` only, so two tasks with different elapsed times compare equal.

`work_labels/domain/entities/work_task.entity.dart`

F8 Stale-unpair guard only ignores one event P2

`_unpairedDeviceIds.remove(bleDevice.id)` consumes the id on the first matching event, so a second stale callback for the same device is processed normally. If MDS can emit more than one, the guard does not hold.

`bluetooth/.../sensor_connection_cubit.dart`

F9 Location is requested but there is no background location permission P2

Sessions can enable location, but the manifest declares only `ACCESS_FINE_LOCATION`. Without `ACCESS_BACKGROUND_LOCATION`, location updates stop as soon as the app is backgrounded on Android 10+. Related to A1 — both are fixed by the same foreground service.

`android/app/src/main/AndroidManifest.xml`

F10 Deep links point at staging P2

Two of the three intent filters are hardcoded to `staging.cloud.wergonic.com` — the mobile login and password-reset links. Only the generic one uses `cloud.wergonic.com`. Confirm production builds handle production links.

`android/app/src/main/AndroidManifest.xml`

Static code read only. No item here has been reproduced on a device. Please verify each one and push back on anything that is intentional.